

The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch27_mlops

AYuSh

Contents

Chapter 27: MLOps & Deployment	3
--------------------------------------	---

Chapter 27: MLOps & Deployment

The previous chapter framed a system on a whiteboard; this one keeps it alive in production. The gap between the two is where most ML value is won or lost, and it is exactly the gap MLOps interviews probe. The uncomfortable truth the field organized itself around — stated in Google's "Hidden Technical Debt in Machine Learning Systems" — is that the model is a small box in a large diagram: the code that trains it is dwarfed by configuration, data collection, feature engineering, serving infrastructure, and monitoring, and every one of those surrounding boxes is a place the system silently rots. A model is not a program you ship once; it is a program whose correctness *decays* because the world it approximates keeps moving. This chapter covers what it takes to deploy and operate one responsibly: the lifecycle and its CI/CD adaptations; the training-serving skew that is the single most common production ML bug, and the feature store that exists to prevent it; model serialization and the serving stack (FastAPI, TorchServe, Triton, ONNX); containerization; the model registry and data/model versioning (MLflow, DVC); the monitoring that catches data drift, concept drift, and degradation; the safe-rollout playbook (shadow, canary, A/B); the choice between batch, real-time, and streaming inference; and the scaling and autoscaling that keep latency under an SLO without burning money.

As in every chapter, the claims are measured, not asserted. The listings are from-scratch simulations of the *operational* mechanisms — not model accuracy, but the behaviour of the system around the model. A training-serving skew study where persisting the exact training transform holds accuracy at 0.828 while recomputing normalization statistics per micro-batch drops it to 0.694, a one-feature unit bug to 0.602, and a stale feature to 0.679 (Listing 1); a serialization comparison where pickling a model and loading it against a version-drifted class definition fails outright (AttributeError) while an ONNX-style computation-graph export reproduces predictions bit-for-bit (max difference 0.0) with no dependency on the training code (Listing 2); a dynamic-batching simulation where serving one request at a time caps a GPU server at 241 req/s — unable to keep up with an 800 req/s load, so latency explodes to 58 seconds — while a batcher waiting up to 5 ms lifts capacity to 1,319 req/s at 8 ms mean latency, and a 20 ms window reaches 2,914 req/s (Listing 3); a drift study where an unsupervised PSI detector climbs 0.002 to 2.06 and KS 0.018 to 0.56 as a feature shifts, crossing the 0.25 "significant" line while labelled accuracy is still only lightly dented — a leading indicator (Listing 4); a concept-drift stream where the input-drift detector stays blind (PSI 0.007 to 0.004) while the error rate jumps 0.216 to 0.655 and a CUSUM monitor on outcomes alarms 32 requests after the true change (Listing 5); a safe-rollout comparison where a blind cutover exposes 100% of users to a regressed challenger, a 5% canary bounds the blast radius to 4.8%, and a shadow deployment incurs zero user harm while still catching the bug (Listing 6); a freshness study where a nightly batch model scores 0.822 because its features are up to 24 hours stale while streaming and real-time serving recover 1.000, tracing the accuracy-

versus-staleness curve that decides the mode (Listing 7); and an M/M/c queueing model where p99 latency detonates from 95 ms at 50% utilization to 622 ms at 98%, and an autoscaler holding 50% utilization meets a 150 ms SLO for 42% fewer replica-hours than a fleet provisioned for peak (Listing 8).

The ML lifecycle and CI/CD for ML

The defining feature of the ML lifecycle is that it is a *loop*, not a line. A traditional software feature is specified, built, tested, shipped, and — barring bugs — done. An ML system is scoped, its data collected and labelled, features engineered, a model trained and evaluated, deployed, and then *monitored so it can be retrained*, because the data distribution that made it accurate will change. The interview signal is that you talk about the whole loop, and especially about the return arrow from monitoring back to data and training, rather than treating deployment as the finish line.

CI/CD adapts to this in one fundamental way: **there are three things under version control, not one — code, data, and model — and the pipeline must test all three.** Continuous integration for ML keeps the unit and integration tests of ordinary software but adds tests that only make sense here: **data validation** (does the incoming batch match the expected schema, ranges, and null rates?), **model validation** against a quality bar (does the retrained model beat the current production model, and — critically — beat it *on every important data slice*, not just in aggregate? A model that improves overall accuracy while regressing on a minority segment must fail the gate), and **behavioural tests** (invariance tests: does perturbing an irrelevant feature leave the prediction unchanged? directional tests: does increasing a feature that should raise the score actually raise it?). Continuous delivery adds the safe-rollout machinery of a later section — you never cut a retrained model straight to 100% of traffic. And because retraining is itself triggered by data (a schedule, a drift alarm, a volume of new labels), mature teams run **continuous training**: an automated pipeline that retrains, validates, and stages a model for rollout without a human writing code, with humans gating the promotion. The recurring interview phrase is that the pipeline, not the model, is the product — the artifact you build once and reuse is the automated path from new data to a validated, deployed model.

Training-serving skew and the feature store

The most common way a model that looked great offline fails in production is not a modelling error at all; it is **training-serving skew** — the features computed at serving time differ, even subtly, from the features the model was trained on. The model is a function; feed it inputs from a different distribution than it learned on and its output is meaningless, no matter how good the model is.

Listing 1 measures the three classic sources on a logistic model whose training features were standardized with the *training* mean and standard deviation. Persisting that exact

transform and applying it at serving is the correct behaviour and holds accuracy at 0.828 under a realistic covariate shift. The failure modes each cost real accuracy.

Recomputing the normalization statistics on each incoming serving batch — a natural-looking choice that is silently wrong, because small batches give noisy estimates and, worse, recomputing *erases* a genuine distribution shift the model was relying on as signal — drops accuracy to 0.694. A **unit or parsing bug on a single feature** (one feature arriving multiplied by 1,000 because an upstream service changed from dollars to cents, or seconds to milliseconds) drops it to 0.602. A **stale feature** — a value that is hours old because the serving path reads a cache the pipeline stopped updating — drops it to 0.679. None of these is a bad model; all of them are the *plumbing* around the model.

The **feature store** exists precisely to make this class of bug structurally impossible. Its defining property is a **single feature-computation path shared by training and serving**: a feature is defined once, and both the offline training job and the online serving path read the *same* definition, so the value a model trains on and the value it serves on are identical by construction. Concretely, a feature store provides an **offline store** (a data warehouse holding historical feature values for building training sets, with correct **point-in-time joins** so a training row only ever sees feature values that existed *before* the label event — preventing the label leakage of joining in a feature computed after the outcome) and an **online store** (a low-latency key-value store, Redis or DynamoDB-class, serving the current feature vector for an entity in single-digit milliseconds at request time). It adds feature **versioning**, **reuse** across teams (the "user's 30-day purchase count" is computed once, not reimplemented in five pipelines each subtly different), and **freshness monitoring**. When an interviewer asks "how do you prevent training-serving skew," the two-word answer is "feature store," and the one-sentence expansion is "compute each feature once from a shared definition and serve the identical value to training and inference, with point-in-time-correct joins offline."

Model serialization and serving

Once a model is trained it has to be **serialized** — turned into an artifact that a serving process can load. The naive choice in Python is to **pickle** the model object, and it is the source of a whole category of production incidents. Pickle stores a reference to the *class*, not the mathematics: unpickling requires the exact class definition and, in practice, compatible library versions, because the pickle re-instantiates a live object whose methods live in code that may have changed. Listing 2 makes the failure concrete: a model pickled under one class definition, loaded in a serving image where an attribute was renamed, fails to load at all (`AttributeError`) — and the more dangerous version of this bug is when it *doesn't* fail but silently mis-predicts because a default filled in for a missing field. Pickle is also a security liability: unpickling executes arbitrary code, so a pickle from an untrusted source is a remote-code-execution vector.

The robust alternative is to serialize the **computation graph** — the operations and their weights — in a framework-independent format, of which **ONNX** (Open Neural Network

Exchange) is the standard. A model exported to ONNX carries its own operator list and weights; a separate, minimal **runtime** (ONNX Runtime) re-executes those operators with no dependency on the training code or even the training framework. Listing 2's from-scratch version of this — a graph of typed ops plus a weights blob, replayed by a tiny runtime that imports nothing from the model class — reproduces the original predictions to floating-point exactness (maximum difference 0.0). This decoupling is the whole point: the team that trains in PyTorch and the team that serves in a C++ or Rust runtime never have to share a Python environment, and the artifact is portable across hardware. Other serialization notes worth having ready: framework-native formats (PyTorch's `state_dict` saved with `torch.save`, TensorFlow's `SavedModel`) are fine *within* a matched environment and are what most training checkpoints use; the `safetensors` format exists specifically to serialize weights *without* pickle's code-execution risk; and for tree models, formats like the booster dumps of XGBoost/LightGBM are the portable path.

Serving the serialized model means wrapping it in something that answers requests, and the stack has a clear ladder. **FastAPI** (or Flask) behind a model is the simplest real option — a Python web service that loads the model and exposes a prediction endpoint; it is perfect for low-to-moderate traffic and full control, and it is what most first deployments are. **TorchServe** and **TensorFlow Serving** are purpose-built model servers that add, out of the box, the things you would otherwise hand-roll: model **versioning** and hot-swapping, multi-model hosting, metrics, and crucially **dynamic batching**. **NVIDIA Triton Inference Server** is the heavyweight, serving models from any framework (ONNX, TensorRT, PyTorch, TensorFlow) with dynamic batching, concurrent model execution on one GPU, and model-ensemble pipelines. The reason model servers earn their complexity over a plain FastAPI wrapper is **dynamic batching**, and Listing 3 quantifies it. A GPU has a large fixed per-call overhead (kernel launch, host-to-device copy, Python) and a tiny marginal cost per item, so serving one request at a time wastes the accelerator catastrophically: at a 4 ms fixed cost the single-request server caps at 241 requests per second and, handed an 800 req/s load, its queue diverges — mean latency 58 *seconds*. A dynamic batcher that waits up to 5 ms to gather up to 8 requests and runs them in one forward pass lifts capacity to 1,319 req/s at 8.3 ms mean latency and a 14 ms p99; widening the window to 20 ms and the batch to 32 reaches 2,914 req/s at 19 ms mean latency. The dial is explicit — a larger batch and a longer wait buy throughput at the cost of latency — and it is the single most important serving-performance knob for deep models.

Containerization for ML

Between the serialized model and the running server sits the **container**. The problem it solves is reproducibility: an ML model depends not just on its weights but on an exact stack — a CUDA version, a cuDNN version, a PyTorch build, a NumPy version, a dozen transitive libraries — and "works on my machine" is the default failure mode when any of them drifts between the training box, the CI runner, and the production node. A **Docker** image freezes the entire userspace stack into an immutable artifact, so the environment

that passed tests is byte-for-byte the environment that serves. The ML-specific practices worth naming: base images matter (the `nvidia/cuda` images or framework-provided images that pin a working CUDA/cuDNN/driver-compatible stack, rather than assembling it by hand); **multi-stage builds** keep the final image small by compiling or downloading in a builder stage and copying only artifacts forward, which matters because ML images bloat into the gigabytes and image size is pull latency at autoscale time; **model weights are usually not baked into the image** but mounted or pulled at startup from object storage or the model registry, so the same image serves many model versions and a new model does not require an image rebuild; and GPU access requires the NVIDIA container runtime. **Kubernetes** is the orchestration layer above containers — it schedules the container replicas across nodes, restarts failed ones, and drives the horizontal autoscaling of the final section; you do not need to administer it to discuss ML deployment, but you should know it is where "run N replicas, keep them healthy, scale them on load" actually happens.

Model registry and versioning

Reproducibility of the *environment* is Docker's job; reproducibility of the *experiment* is the registry's and the version-control system's. The governing principle is that an ML result is a function of three inputs — **code, data, and model (plus hyperparameters and environment)** — and reproducing or auditing a model requires versioning all of them together. Git versions the code and configuration well and versions large binary data and model files badly, which is the gap the ML tooling fills.

MLflow is the common answer for the model side. Its Tracking component logs every training run — parameters, metrics, the code version, and the output model artifact — so experiments are comparable and reproducible rather than lost in notebook output. Its **Model Registry** is the system of record for models headed to production: a trained model is registered as a named, versioned entry and moves through explicit **stages** (Staging, Production, Archived) with an audit trail of who promoted what and when. This is what makes **rollback** a first-class operation — production is a pointer to a registry version, and rolling back is repointing it at the previous known-good version, not a redeploy scramble. The registry is also where model **lineage** lives: which data and which run produced this artifact.

DVC (Data Version Control) is the common answer for the data side. It versions large datasets and model files by storing a small hash-pointer file *in Git* while the actual bytes live in object storage (S3, GCS), so `git checkout` of an old commit brings back not just the old code but the exact dataset and model that commit was built with. It also expresses **pipelines** as dependency graphs (raw data to features to model), re-running only the stages whose inputs changed. The interview synthesis: Git for code, DVC (or a feature store's offline snapshots, or a lakehouse's time-travel) for data, and a model registry (MLflow) for models and their promotion lifecycle — together they make any deployed model reproducible from a single commit hash and auditable end to end.

Monitoring: data drift, concept drift, and degradation

A deployed model has no natural alarm when it becomes wrong. Unlike a crashed service it keeps returning confident predictions; it just returns worse ones. Monitoring is what converts silent decay into a signal, and the interview distinction that matters is **what kind of decay** you are catching, because the detectors differ.

Data drift (covariate shift) is a change in the input distribution $P(x)$ — the users, items, or transactions arriving now look different from those the model trained on. It is detectable **without labels**, which is its great operational virtue, because labels are usually delayed. The standard detectors compare the current feature distribution to a training reference: the **Population Stability Index (PSI)**, a symmetrized divergence over fixed training-quantile bins with a widely used rule of thumb (below 0.1 stable, 0.1 to 0.25 moderate, above 0.25 significant), and the **two-sample Kolmogorov-Smirnov** statistic for continuous features, plus per-feature summary-statistic and null-rate monitors for the cheap-but-effective first line. Listing 4 drifts one feature progressively and shows both detectors rising monotonically — PSI 0.002 to 2.06, KS 0.018 to 0.56 — and crossing their thresholds while the labelled accuracy has only begun to erode (PSI passes 0.25 when accuracy has fallen from 0.763 to 0.723). That ordering is the point: input-drift monitoring is a **leading indicator** you can act on before ground-truth labels arrive to confirm the damage. The honest caveat, which strong candidates volunteer, is that covariate shift does not *always* degrade accuracy — if $P(y|x)$ is unchanged and the model generalizes, the input distribution can move without hurting — so a drift alarm is a prompt to investigate, not proof of degradation.

Concept drift is a change in the relationship $P(y|x)$ itself — the same inputs now map to different outcomes (spam tactics evolve, fraud patterns adapt, purchasing behaviour shifts after a shock). This is the dangerous kind, and Listing 5 shows why: it is **invisible to input-drift detectors**. In a stream whose decision boundary rotates midway while the input distribution is held fixed, PSI on the inputs reads 0.007 before and 0.004 after the change — nothing — while the true error rate jumps from 0.216 to 0.655. Catching it requires a signal derived from **outcomes**: the model's error rate once labels arrive, or a proxy for it. The listing runs a **CUSUM** sequential detector on the error stream — accumulating evidence that the error rate has risen above its pre-drift baseline and alarming when the cumulative sum crosses a threshold — and it fires 32 requests after the true change, where a per-window control chart would have drowned in false alarms. The named detectors in this family are **DDM/EDDM** (drift detection on the error rate) and **ADWIN** (an adaptive window that shrinks when a change is detected); the practical constraint behind all of them is **label latency** — if labels take weeks (fraud chargebacks, churn), concept-drift detection is weeks-delayed, which is why teams also monitor label-free proxies like the drift of the model's *output* score distribution and the drift of its inputs, and why human-labelled sample streams are worth paying for.

Beyond drift, production monitoring watches **prediction distribution** (has the average score moved? has the rate of a given class shifted?), **operational metrics** (latency p50/

p99, throughput, error rate, resource saturation — the SRE layer), **data-quality metrics** (nulls, schema violations, out-of-range values, feature freshness), and **slice metrics** (accuracy on the segments that matter, so an aggregate that hides a subgroup collapse gets caught). The mature setup ties an alarm to an action: a drift alarm opens an investigation and, if confirmed, triggers retraining and a staged rollout — closing the lifecycle loop.

Safe deployment: shadow, canary, and A/B

No validated-offline model ships straight to all traffic, because offline validation cannot see the failures that only appear under real load with real features — the training-serving skew of Listing 1 among them. The deployment playbook is a ladder of progressively riskier exposure, each rung buying different information.

Shadow deployment (dark launch) runs the new model alongside the current one, feeding it a copy of every live request, logging its predictions, and comparing them to production — but **never serving its output to users**. Its defining property, measured in Listing 6, is **zero user harm**: because no user ever sees a shadow prediction, a broken challenger costs nothing while you catch it. Shadow mode validates that the serving path works end to end (the model loads, features resolve, latency is acceptable) and that predictions are sane relative to the incumbent, and once labels arrive it measures the accuracy regression directly. Its limitation is equally defining: it cannot measure the **behavioural response** — because users never act on shadow predictions, you learn nothing about whether the new model would change clicks, conversions, or revenue.

Canary deployment serves the new model to a **small slice of real traffic** (1–5%) behind close monitoring, with automatic rollback on a regression. Its defining property is a **bounded blast radius**: Listing 6 holds it to 4.8% of users even as a regressed challenger is caught, versus 100% for a blind cutover. The listing's honest secondary finding is worth stating because interviewers probe it — the *count* of extra errors served before rollback is similar for canary and blind cutover (20 versus 18), because the number of bad predictions needed to *detect* the regression is set by the statistical detector, not the traffic split. What the canary buys is not fewer total errors on the challenger arm but a cap on how much of the userbase is exposed **at once** — which is exactly what protects you against the failures a metric cannot pre-see (a crash loop, a latency blowup, an outage), where you want 5% of users degraded, not everyone.

A/B testing is the top of the ladder and the arbiter of *value*: randomize users into control (current model) and treatment (new model), serve each its own system, and compare online business metrics with a significance test. It is the only rung that answers "is the new model actually better for the business," and the statistical machinery — sample size, power, guardrail metrics, A/A validation — is the same as Chapter 26's, whose power calculation showed how many users a small relative lift needs. The three rungs compose in practice: shadow to prove the plumbing, canary to bound operational risk while

gathering live feedback, A/B to prove value, then a **progressive rollout** (5% to 25% to 50% to 100%) with the monitoring of the previous section watching the whole way and a registry rollback pointer ready.

Batch, real-time, and streaming inference

Not every model serves the same way, and choosing the mode is a design decision with a measurable consequence: **feature freshness**. Listing 7 makes the tradeoff concrete on a signal whose true state random-walks over a day so the correct label depends on the *current* state.

Batch inference precomputes predictions on a schedule (nightly, hourly) from a data snapshot and stores them for lookup. It is the cheapest and operationally simplest mode — one job, no serving latency, trivial to scale — and it is correct whenever predictions do not need to reflect within-day events: a next-day churn score, a weekly recommendation refresh, a nightly risk re-rating. Its cost is **staleness**: in the listing, a model whose features are the midnight snapshot scores 0.822 because by afternoon those features are up to 24 hours old.

Real-time (online) inference computes a prediction on demand at request time with the freshest possible features, recovering accuracy to 1.000 in the listing but paying for it — a low-latency serving path (the FastAPI/Triton stack, tens of milliseconds), an online feature store, and roughly 20× the per-request compute of a lookup. It is mandatory when the prediction depends on request-time context (a fraud decision on *this* transaction, a search ranking for *this* query) that no precomputation could have anticipated.

Streaming inference is the middle ground: an event pipeline (Kafka, Flink, Spark Structured Streaming) updates features continuously as events arrive, so predictions use features that are seconds-to-minutes fresh without a full request-time recomputation. In the listing its ~15-minute-fresh features also recover essentially full accuracy at a fraction of real-time's per-request cost. The staleness sweep is the general lesson — accuracy 1.000 at 0 hours stale, 0.921 at 2 hours, 0.844 at 12, 0.822 at 24 — so the required freshness of the decision, not fashion, selects the mode: batch where a day-old feature is fine, streaming where minutes matter, real-time where the request itself carries the signal. A common production pattern is hybrid — precompute the expensive features in batch or streaming, compute only the cheap request-time features online, and combine them at serving.

Scaling and autoscaling

The last operational concern is capacity, and it is governed by one counterintuitive fact from queueing theory: **latency does not degrade gracefully as you approach full utilization — it detonates**. Listing 8's M/M/c model of a four-replica server makes the

wall visible: at 50% utilization p99 latency is 95 ms, at 70% it is 97 ms, but at 85% it jumps to 164 ms, at 92% to 275 ms, and at 98% to 622 ms. The $1/(1-p)$ blowup means you *cannot* plan to run a server near 100% utilization; the headroom that looks like waste is what keeps the tail latency bounded, and the target utilization is a direct consequence of your tail-latency SLO and the variance of your service time.

Scaling comes in two directions. **Vertical scaling** — a bigger machine, more GPU memory, a faster accelerator — is how you fit a larger model or a larger batch, and it has a ceiling (the biggest available instance) and a single-point-of-failure profile. **Horizontal scaling** — more replicas behind a load balancer — is how you serve more traffic, and it is the default for stateless serving because it scales nearly linearly and adds redundancy. GPU serving adds its own wrinkles: GPUs are expensive so you want high utilization, which is exactly what dynamic batching (Listing 3) and concurrent model execution (Triton) provide, and techniques like the KV-cache paging and continuous batching of Chapter 22 are the LLM-specific expression of the same pressure.

Autoscaling closes the loop by adjusting replica count to load automatically. The critical design choice is the **scaling signal**: CPU utilization is the Kubernetes default and is nearly useless for GPU-bound or IO-bound model serving; the signals that actually track the SLO are **request queue depth**, **latency**, or **GPU utilization**, often via a custom metric. Listing 8 runs an autoscaler that adds and removes replicas to hold utilization near 50% through a diurnal load curve swinging from 30 to 210 req/s, and it meets a 150 ms p99 SLO with zero violations for **42% fewer replica-hours** (125 versus 216) than a fixed fleet provisioned for the peak — the standard autoscaling win, matching supply to demand instead of paying for peak all day. The operational caveats to name: a cold model replica is slow to start (image pull plus model load plus GPU warm-up can be minutes), so autoscalers need **headroom and predictive or scheduled scaling** to absorb spikes the reactive loop is too slow for; scaling to zero saves money for spiky low-volume models but pays a cold-start latency tax on the first request; and every autoscaler needs a **maximum** to bound cost and a **minimum** to hold a warm floor.

Code implementations

(Each listing is a self-contained from-scratch simulation of one operational mechanism over controlled synthetic data, so the systems claim — not a particular dataset or model — is what is measured. The numbers quoted in the prose are these programs' actual output.)

Listing 1 — Training-serving skew: why the plumbing breaks the model

The correct behaviour is to persist the exact training transform. Recomputing statistics at serving, a unit bug on one feature, and a stale feature each cost real accuracy — none of them a modelling error. This is the argument for a feature store.

```

"""Listing 1: training-serving skew. A logistic model is trained on features
standardized with
TRAIN statistics. At serving four things can go wrong: (a) recomputing mean/std per
micro-batch
instead of persisting the training stats (small batches -> noisy stats, and a real
covariate shift
gets erased); (b) a unit/parsing bug on one feature; (c) a stale feature (hours old).
We measure
the accuracy each induces versus 'persist the exact training transform'. This is the
classic
production ML bug and the core argument for a feature store: compute a feature ONCE,
serve the
identical value at train and serving time."""
import numpy as np
rng=np.random.default_rng(0)
n,d=8000,6
w_true=np.array([1.6,-1.1,0.9,0.0,0.6,-0.8])
def make(n,shift):
    X=rng.normal(0,1,(n,d))+shift
    y=(rng.random(n)<1/(1+np.exp(-(X@w_true-0.3))))).astype(int)
    return X,y
Xtr,ytr=make(n,0.0)
mu,sd=Xtr.mean(0),Xtr.std(0) # TRAIN statistics – the artifact
to persist
def fit(Xn,y,epochs=300,lr=0.3):
    w=np.zeros(Xn.shape[1]); b=0.0
    for _ in range(epochs):
        p=1/(1+np.exp(-(Xn@w+b))); g=p-y
        w-=lr*(Xn.T@g/len(y)); b-=lr*g.mean()
    return w,b
w,b=fit((Xtr-mu)/sd,ytr)
pred=lambda Xn:(1/(1+np.exp(-np.clip(Xn@w+b,-30,30)))>0.5)
shift=np.array([0.6,-0.4,0.3,0.0,0.2,-0.3]) # real per-feature covariate shift
at serving
Xte,yte=make(4000,shift)
# (a) persist train transform (b) recompute per 32-row micro-batch
acc_correct=pred((Xte-mu)/sd)==yte
def recompute_microbatch(X):
    out=np.empty_like(X)
    for i in range(0,len(X),32):
        b=X[i:i+32]; out[i:i+len(b)]=(b-b_.mean(0))/(b_.std(0)+1e-9)
    return out
acc_recompute=pred(recompute_microbatch(Xte))==yte
Xu=Xte.copy(); Xu[:,4]=1000 # (c) feature 4 arrives in wrong
unit
acc_unit=pred((Xu-mu)/sd)==yte
Xs=Xte.copy(); Xs[:,0]=Xtr[:4000,0] # (d) feature 0 stale: serves an
old snapshot
acc_stale=pred((Xs-mu)/sd)==yte
print(f"correct (persisted transform)      acc = {acc_correct.mean():.3f}")
print(f"recompute stats per micro-batch    acc = {acc_recompute.mean():.3f}")
print(f"unit bug on one feature (x1000)      acc = {acc_unit.mean():.3f}")
print(f"stale feature (old snapshot)          acc = {acc_stale.mean():.3f}")

```

Listing 2 — Serialization: pickle fragility vs a portable computation graph

Pickling stores a reference to the class, not the math, and breaks under version drift. An ONNX-style graph export replays the operators in a standalone runtime, reproducing predictions bit-for-bit with no dependency on the training code.

```

"""Listing 2: model serialization. Two ways to persist a trained model. (1) PICKLE the live Python object: compact, but it stores a reference to the CLASS, not the math – load it in an environment where the class definition changed (a renamed attribute, a bumped library version) and it breaks or, worse, silently mis-predicts. (2) Export the COMPUTATION GRAPH (weights + op list) to a framework-independent format (ONNX is the standard): a separate minimal runtime re-executes the ops with no dependency on the training code. We show the exported graph reproduces predictions to float precision, that the runtime needs none of the training class, and that a version drift silently corrupts the pickle path."""
import numpy as np, pickle, io
rng=np.random.default_rng(0)
X=rng.normal(0,1,(2000,8)); w1=rng.normal(0,.5,(8,16)); b1=rng.normal(0,.1,16)
w2=rng.normal(0,.5,(16,3)); b2=rng.normal(0,.1,3)
def relu(z): return np.maximum(0,z)
def softmax(z): z=z-z.max(1,keepdims=True); e=np.exp(z); return e/e.sum(1,keepdims=True)

class MLP: # the "training" class, version 1
    def __init__(s,w1,b1,w2,b2): s.w1,s.b1,s.w2,s.b2=w1,b1,w2,b2
    def predict(s,X): return softmax(relu(X@s.w1+s.b1)@s.w2+s.b2)
m=MLP(w1,b1,w2,b2); ref=m.predict(X)

# --- path 1: pickle the object ---
blob=pickle.dumps(m)
class MLP_v2: # v2 in the serving image: someone renamed w2 -> W2
    def __init__(s,w1,b1,W2,b2): s.w1,s.b1,s.W2,s.b2=w1,b1,W2,b2
    def predict(s,X): # references s.W2; pickle restored s.w2 -> breaks
        return softmax(relu(X@s.w1+s.b1)@s.W2+s.b2)
import __main__; __main__.MLP=MLP_v2
# unpickle resolves the name to the NEW class
try:
    m2=pickle.loads(blob); out=m2.predict(X)
    print(f"pickle across version drift: max|diff| = {np.abs(out-ref).max():.3e} (silently served)")
except Exception as e:
    print(f"pickle across version drift: FAILED to load -> {type(e).__name__}")

# --- path 2: export the graph (weights + typed op list), re-run in a tiny standalone runtime ---
graph={"nodes": [("matmul", "w1"), ("add", "b1"), ("relu", None), ("matmul", "w2"), ("add", "b2"), ("softmax", None)],
        "weights": {"w1":w1, "b1":b1, "w2":w2, "b2":b2}}
buf=io.BytesIO(); np.savez(buf,**{f"W::{k}":v for k,v in graph["weights"].items()});
meta=graph["nodes"] # (in ONNX both live in one .onnx protobuf)
def runtime(X,nodes,W): # depends on nothing from the training code
    h=X
    for op,arg in nodes:
        if op=="matmul": h=h@W[arg]
        elif op=="add": h=h+W[arg]
        elif op=="relu": h=np.maximum(0,h)
        elif op=="softmax": h=softmax(h)
    return h
buf.seek(0); loaded={k.split("::")[1]:v for k,v in np.load(buf).items()}
out2=runtime(X,meta,loaded)
print(f"graph export (ONNX-style): max|diff| = {np.abs(out2-ref).max():.3e} (bit-for-bit)")

```

Listing 3 — Dynamic batching: the core serving-throughput knob

One request at a time wastes the accelerator's fixed per-call overhead and caps capacity below the offered load, so latency diverges. A dynamic batcher trades a small wait window for a large throughput gain — the mechanism inside TorchServe, Triton, and vLLM.

```

"""Listing 3: dynamic batching for online serving. Requests arrive as a Poisson stream.
A GPU model
has a fixed per-call overhead (kernel launch, Python, H2D copy) plus a tiny marginal
cost per item,
so serving one request at a time wastes the accelerator: at bs=1 each call costs
OVERHEAD+PER_ITEM,
capping capacity far below the offered load, so the queue and latency explode. A
dynamic batcher
waits up to `max_delay` ms to accumulate up to `max_batch` requests, then runs them in
one forward
pass – the mechanism inside TorchServe/Triton/vLLM. We measure server capacity, mean
batch size,
and mean + p99 latency under a fixed 800 req/s load. Batching multiplies capacity; a
small wait
window buys most of it at little latency cost."""
import numpy as np
rng=np.random.default_rng(0)
LOAD=800.0; OVERHEAD=4.0; PER_ITEM=0.15; T=40000
gap=rng.exponential(1000.0/LOAD,T); arrivals=np.cumsum(gap)
def simulate(max_batch,max_delay):
    server_free=0.0; i=0; lat=[]; bsz=[]
    while i<T:
        open_t=max(server_free,arrivals[i])                # batch opens
        j=i+1
        while j<T and arrivals[j]<=open_t+max_delay and (j-i)<max_batch: j+=1
        bs=j-i; svc=OVERHEAD+PER_ITEM*bs
        start=max(open_t,arrivals[j-1])                    # can't run until all
        admitted have arrived
        finish=start+svc
        for r in range(i,j): lat.append(finish-arrivals[r])
        bsz.append(bs); server_free=finish; i=j
    lat=np.array(lat)
    cap=1000.0*len(lat)/ (np.array([OVERHEAD+PER_ITEM*b for b in bsz]).sum()) # req
    per busy-second
    return cap,np.mean(bsz),lat.mean(),np.percentile(lat,99)
print(f"offered load = {LOAD:.0f} req/s | overhead {OVERHEAD} ms/call, {PER_ITEM} ms/
item\n")
print(f"{'policy':30s}{{'capacity':>11s}{{'mean bs':>9s}{{'mean lat':>10s}{{'p99':>9s}}}}")
for mb,md,name in [(1,0,"no batching (bs=1)"),(8,5,"batch<=8, wait<=5ms"),
(32,5,"batch<=32, wait<=5ms"),(32,20,"batch<=32, wait<=20ms"),
(128,20,"batch<=128, wait<=20ms")]:
    cap,mbs,ml,p99=simulate(mb,md)
    flag=" <- cannot keep up" if cap<LOAD else ""
    print(f"{'name':30s}{cap:8.0f}/s{mbs:8.1f}{ml:8.1f}ms{p99:7.1f}ms{flag}")

```

Listing 4 — Data drift detection: PSI and KS as leading indicators

Unsupervised detectors that need no labels. Both rise monotonically as a feature drifts and cross their thresholds before ground-truth accuracy confirms the damage.

```

"""Listing 4: data (covariate) drift detection. The serving input distribution moves
away from
training over time. Two standard unsupervised detectors run WITHOUT labels: the


```



```

Population Stability
Index (PSI) over fixed training-quantile bins, and the two-sample Kolmogorov-Smirnov
statistic.
Industry PSI rule of thumb: <0.1 stable, 0.1-0.25 moderate, >0.25 significant. We drift
one feature
progressively and show both detectors rise monotonically and cross their thresholds
BEFORE (and
predictive of) the labelled-accuracy decay a monitor would otherwise only catch weeks
later when
ground truth arrives."""
import numpy as np
rng=np.random.default_rng(0)
d=5; w_true=np.array([1.5,-1.0,0.8,0.6,-0.7])
def label(X):
    z=X@w_true-0.2-0.9*(X[:,0]**2)    # nonlinear in feature 0 (linear model can't see
it)
    return (rng.random(len(X))<1/(1+np.exp(-np.clip(z,-30,30))))).astype(int)
Xtr=rng.normal(0,1,(20000,d)); ytr=label(Xtr)
mu,sd=Xtr.mean(0),Xtr.std(0)
def fit(Xn,y,epochs=300,lr=0.3):
    w=np.zeros(d); b=0.0
    for _ in range(epochs):
        p=1/(1+np.exp(-np.clip(Xn@w+b,-30,30))); g=p-y
        w-=lr*(Xn.T@g/len(y)); b-=lr*g.mean()
    return w,b
w,b=fit((Xtr-mu)/sd,ytr)
acc=lambda X,y:((1/(1+np.exp(-np.clip(((X-mu)/sd)@w+b,-30,30)))>0.5)==y).mean()
# reference bins for PSI: deciles of the training values of feature 0
edges=np.quantile(Xtr[:,0],np.linspace(0,1,11)); edges[0]=-np.inf; edges[-1]=np.inf
ref=np.histogram(Xtr[:,0],edges)[0]/len(Xtr)
def psi(cur):
    c=np.histogram(cur,edges)[0]/len(cur); c=np.clip(c,1e-4,None); r=np.clip(ref,
1e-4,None)
    return float(((c-r)*np.log(c/r)).sum())
def ks(cur):
    # two-sample KS vs a training reference
sample
    a=np.sort(Xtr[:5000,0]); b_=np.sort(cur)
    grid=np.concatenate([a,b_])
    Fa=np.searchsorted(a,grid,'right')/len(a); Fb=np.searchsorted(b_,grid,'right')/
len(b_)
    return float(np.abs(Fa-Fb).max())
print(f"{'feature-0 shift':>16s}{'PSI':>8s}{'KS':>7s}{'accuracy':>10s}    verdict")
for s in [0.0,0.25,0.5,0.75,1.0,1.5]:
    Xc=rng.normal(0,1,(8000,d)); Xc[:,0]+=s
    yc=label(Xc); p=psi(Xc[:,0]); k=ks(Xc[:,0]); a=acc(Xc,yc)
    v="stable" if p<0.1 else ("MODERATE" if p<0.25 else "SIGNIFICANT drift")
    print(f"{s:14.2f}    {p:7.3f}{k:7.3f}{a:9.3f}    {v}")

```

Listing 5 — Concept drift: invisible to input monitors, caught on outcomes

The decision boundary rotates while the input distribution holds fixed. PSI on the inputs sees nothing; only a CUSUM monitor on the error stream catches it, shortly after the true change.

```

"""Listing 5: concept drift vs data drift. Concept drift = P(y|x) changes while the
input
distribution P(x) stays fixed. An unsupervised input-drift detector (PSI, Listing 4) is
BLIND to it
because the inputs never move; only a labelled signal catches it. We stream data whose
decision
boundary rotates at t=4000, hold P(x) constant, and run two monitors: PSI on the inputs
(stays flat,

```



```

sees nothing) and a DDM-style monitor on the running error rate (fires). We monitor the
error stream with a CUSUM
sequential detector:  $S = \max(0, S + (\text{err} - (p_0 + \text{slack})))$  accumulates evidence that the
error rate
has risen above the pre-drift baseline  $p_0$ , alarming when  $S$  crosses a threshold  $h$ . CUSUM
is robust to
single-window noise (unlike a per-window control chart, which false-alarms over
thousands of
windows). Lesson: monitor predictions and outcomes, not just inputs."""
import numpy as np
rng=np.random.default_rng(1)
d=4
def boundary_labels(X,w,b): return (rng.random(len(X))<1/(1+np.exp(-np.clip(X@w+b,-30,
30))))).astype(int)
w0=np.array([1.4,-1.1,0.7,0.5]); b0=-0.2
Xtr=rng.normal(0,1,(12000,d)); ytr=boundary_labels(Xtr,w0,b0)
def fit(X,y,ep=250,lr=0.3):
    w=np.zeros(d);b=0.0
    for _ in range(ep):
        p=1/(1+np.exp(-np.clip(X@w+b,-30,30)));g=p-y
        w-=lr*(X.T@g/len(y));b-=lr*g.mean()
    return w,b
w,b=fit(Xtr,ytr)
edges=np.quantile(Xtr[:,0],np.linspace(0,1,11));edges[0]=-np.inf;edges[-1]=np.inf
ref=np.clip(np.histogram(Xtr[:,0],edges)[0]/len(Xtr),1e-4,None)
def psi(x):
    c=np.clip(np.histogram(x,edges)[0]/len(x),1e-4,None); return float(((c-
ref)*np.log(c/ref)).sum())
# stream 8000 requests; P(x) fixed throughout; at t=4000 the LABEL rule rotates
(concept drift)
N=8000; Xs=rng.normal(0,1,(N,d))
w1=np.array([-0.3,1.3,-0.9,0.6]); b1=0.1 # new concept
ys=np.where(np.arange(N)<4000, boundary_labels(Xs,w0,b0), boundary_labels(Xs,w1,b1))
pred=(1/(1+np.exp(-np.clip(Xs@w+b,-30,30)))>0.5).astype(int)
err=(pred!=ys).astype(float)
# DDM over the error stream
p0=err[:2000].mean()
# pre-drift baseline error (from a warm-up window)
slack=0.20; h=8.0 # slack: only accumulate error above baseline+0.20; h:
alarm level
S=0.0; drift=None
for i in range(2000,N):
    S=max(0.0,S+(err[i]-(p0+slack)))
    if S>h: drift=i; break
print(f"PSI on inputs during stream: {psi(Xs[:4000,0]):.3f} (pre) / {psi(Xs[4000:,0]):.
3f} (post) -> input detector BLIND")
print(f"error rate: pre-drift {err[:4000].mean():.3f} post-drift {err[4000:].mean():.
3f}")
print(f"CUSUM DRIFT alarm at request {drift} (true change at 4000, detection lag
{drift-4000} requests)")

```

Listing 6 — Safe rollout: shadow and canary blast radius

A challenger that looked better offline is worse online. Blind cutover exposes 100% of users, a 5% canary bounds the blast radius, and shadow deployment incurs zero user harm while still catching the regression.

""""Listing 6: safe rollout – shadow and canary. A challenger looks better OFFLINE on stale logs but is actually WORSE online (it inherited a training-serving skew bug). A rollout runs a fixed soak window (observe the challenger arm for T requests, then decide) with a CUSUM auto-rollback that can cut the soak short if the regression is severe. We compare three strategies on (i)

```

blast radius –
fraction of users who ever touch the challenger – and (ii) realized regret, the extra
errors served
versus staying on the champion:
- Blind cutover: 100% to challenger. Blast radius 100%.
- Canary 5%: 5% of traffic to challenger behind the same monitor. Blast radius ~5%.
- Shadow: challenger scored on a copy of every request, NEVER served. Blast radius 0 –
but it can
    only catch what is measurable without serving (prediction divergence, latency,
    crashes, and
    delayed-label accuracy), not the online behavioural response a canary measures."""
import numpy as np
rng=np.random.default_rng(3)
N=120000; champ_err=0.10; chall_err=0.16      # subtle regression: offline missed it
T=60000                                     # soak window (challenger-arm requests
before decision)
print(f"offline (stale logs): champion 0.11 challenger 0.08 -> challenger promoted")
print(f"true online error:   champion {champ_err:.2f} challenger {chall_err:.2f}\n")
def cusum_rollback(errs,p0=champ_err,slack=0.03,h=10.0):
    S=0.0
    for i,e in enumerate(errs):
        S=max(0.0,S+(e-(p0+slack)))
        if S>h: return i+1
    return len(errs)
def run(canary_frac):
    is_canary=rng.random(N)<canary_frac
    canary_pos=np.where(is_canary)[0]
    errs_challenger=(rng.random(len(canary_pos))<chall_err).astype(int)
    stop=min(T,cusum_rollback(errs_challenger))      # decide at soak end or CUSUM
alarm
    served=canary_pos[:stop]
    total_reqs_elapsed=served[-1]+1 if len(served) else 0
    extra=int(errs_challenger[:stop].sum()-round(champ_err*stop))
    blast=len(served)/max(1,total_reqs_elapsed)
    return stop,extra,blast,total_reqs_elapsed
for name,f in [("blind cutover (100%)",1.0),("canary 5%",0.05)]:
    stop,extra,blast,elapsed=run(f)
    print(f"{name:22s} rolled back after {stop:4d} challenger reqs ({elapsed:6d} total
elapsed), "
        f"blast radius {blast*100:4.1f}%, extra errors served = {extra}")
print(f"{'shadow (0% served)':22s} caught by logged divergence, blast radius 0.0%,
extra errors served = 0")

```

Listing 7 — Batch vs real-time vs streaming: freshness selects the mode

Feature staleness, not model quality, separates the serving modes. The accuracy-versus-staleness curve is the quantity that decides how fresh your features must be — and therefore how much infrastructure you buy.

```

"""Listing 7: batch vs real-time vs streaming inference. The three serving modes differ
in FRESHNESS
of the features they use. We simulate a churn/fraud-style signal where each user's
latent state
random-walks over the day and the label depends on the CURRENT state. Three modes:
- Batch: predictions precomputed nightly from the 00:00 snapshot, served all day -> up
to 24h stale.
- Streaming: features updated by an event pipeline every ~15 min -> effectively fresh.
- Real-time: features computed at request time -> zero staleness, highest per-request
cost/latency.
We measure accuracy vs feature staleness, paired with the cost/latency each mode
implies. Lesson:
freshness is a business requirement that SELECTS the serving mode; you pay for it in

```

```

infra."""
import numpy as np
rng=np.random.default_rng(0)
users,hours=6000,24
steps=rng.normal(0,0.25,(hours,users)); steps[0]=rng.normal(0,1,users)
state=np.cumsum(steps,axis=0)
# state[h,u] = latent state of user u at hour h
label=lambda s:(s>0.5).astype(int)
w,b=1.0,-0.5
predict=lambda s:(1/(1+np.exp(-np.clip(w*s+b,-30,30)))>0.5).astype(int)
req_hour=rng.integers(0,hours,users) # each user is queried once at a random
hour
true_now=label(state[req_hour,np.arange(users)])
def acc_at_staleness(h):
    src=np.clip(req_hour-h,0,hours-1)
    return (predict(state[src,np.arange(users)])==true_now).mean()
batch_acc=(predict(state[0,np.arange(users)])==true_now).mean()
# nightly snapshot at hour 0
print(f"{'mode':12s}{'staleness':>13s}{'accuracy':>10s}{'rel infra cost':>17s}
{'latency':>12s}")
print(f"{'batch':12s}{'up to 24h':>13s}{batch_acc:9.3f}{'1x nightly':>17s}{'~0
lookup':>12s}")
print(f"{'streaming':12s}{'~15 min':>13s}{acc_at_staleness(0):9.3f}{'~5x event
pipe':>17s}{'~0 lookup':>12s}")
print(f"{'real-time':12s}{'0 on-demand':>13s}{acc_at_staleness(0):9.3f}{'~20x per-
req':>17s}{'10-50 ms':>12s}")
print("\n feature-staleness sweep (accuracy vs how old the feature is):")
for h in [0,2,6,12,24]:
    print(f"    {h:2d}h stale: accuracy {acc_at_staleness(min(h,hours-1)):.3f}")

```

Listing 8 — Scaling: the utilization wall and autoscaling

Tail latency detonates as utilization approaches 1, so you cannot run near capacity. An autoscaler that holds a target utilization meets the SLO through a diurnal load swing for far fewer replica-hours than a fleet provisioned for peak.

```

"""Listing 8: scaling and autoscaling. A model server has C replicas, each serving one
request at a
time at a fixed service rate. Requests arrive as a Poisson stream. Two facts drive
every capacity
decision: (1) tail latency explodes NONLINEARLY as utilization rho = load/(C*rate)
approaches 1 -
the classic 1/(1-rho) queueing wall, so you cannot run near 100% utilization; (2) a
horizontal
autoscaler that adds/removes replicas to keep utilization in a target band holds p99
latency under an
SLO through a daily load swing, which a fixed fleet cannot. We simulate an M/M/c queue
and measure
p50/p99 vs rho, then run an autoscaler against a diurnal load curve."""
import numpy as np, heapq
rng=np.random.default_rng(0)
RATE=50.0 # requests/sec each replica can serve
def mmc_latency(load,C,T=60.0):
    # event-driven M/M/c: returns p50, p99 latency (ms) and utilization
    n=int(load*T); arr=np.cumsum(rng.exponential(1.0/load,n))
    free=[0.0]*C; heapq.heapify(free); lat=[]
    for t in arr:
        r=heapq.heappop(free); start=max(t,r)
        svc=rng.exponential(1.0/RATE); done=start+svc
        lat.append((done-t)*1000); heapq.heappush(free,done)
    lat=np.array(lat); return np.percentile(lat,50),np.percentile(lat,99),load/(C*RATE)
print("Tail latency vs utilization (C=4 replicas, 50 req/s each = 200 capacity):")
print(f"{'load':>7s}{'util':>7s}{'p50':>9s}{'p99':>10s}")
for load in [100,140,170,185,195]:

```

```

p50,p99,rho=mmc_latency(load,4)
wall="  <- wall" if p99>200 else ""
print(f"{load:7d}{rho:6.0%}{p50:8.1f}ms{p99:9.1f}ms{wall}")
# --- autoscaler vs a fleet provisioned for PEAK, over a diurnal load curve ---
SLO=150.0                                # p99 latency SLO (ms)
hours=np.arange(24)
load_curve=(120+90*np.sin((hours-8)/24*2*np.pi)).clip(30)  # 30..210 req/s, peaks
midday
peakC=int(np.ceil(load_curve.max()/(RATE*0.5)))             # fixed fleet sized for
peak @ 50% util
def autoscale(target_util=0.5):                             # add/remove replicas to hold
utilization ~50%
    hist=[]
    for L in load_curve:
        C=max(1,int(np.ceil(L/(RATE*target_util))))
        _,p99,_=mmc_latency(L,C); hist.append((C,p99))
    return hist
fx=[(peakC,mmc_latency(L,peakC)[1]) for L in load_curve]
au=autoscale()
au_viol=sum(p99>SLO for _,p99 in au); fx_viol=sum(p99>SLO for _,p99 in fx)
au_cost=sum(c for c,_ in au); fx_cost=sum(c for c,_ in fx)
print(f"\nDiurnal load 30-210 req/s, SLO p99<{SLO:.0f}ms:")
print(f"  fixed fleet sized for peak (C={peakC}): {fx_viol:2d}/24 h violated, replica-
hours = {fx_cost}")
print(f"  autoscaler (target 50% util)      : {au_viol:2d}/24 h violated, replica-hours
= {au_cost}  ({100*(1-au_cost/fx_cost):.0f}% cheaper)")

```

Questions & Answers

Q1. What is MLOps, and why is the ML lifecycle a loop rather than a line?

MLOps is the set of practices for reliably deploying and operating ML systems: versioning, CI/CD, serving, monitoring, and retraining. It is a loop because a model's accuracy decays as the data distribution it approximates changes, so monitoring feeds back into data collection and retraining rather than deployment being the finish line. The interview signal is discussing the return arrow from monitoring to retraining, and treating the automated pipeline — not the model — as the artifact you build.

Q2. How does CI/CD for ML differ from CI/CD for ordinary software?

Three things are under version control, not one — code, data, and model — and the pipeline must test all three. Beyond ordinary unit/integration tests it adds data validation (schema, ranges, null rates), model validation against a quality bar including per-slice checks (a model that improves in aggregate but regresses on a subgroup must fail the gate), and behavioural tests (invariance and directional). Continuous delivery never cuts straight to 100% traffic; it uses the shadow/canary/A-B ladder.

Q3. What is continuous training and how does it relate to CI/CD?

Continuous training is an automated pipeline that retrains, validates, and stages a model when triggered by data — a schedule, a drift alarm, or a volume of new labels — without a human writing code, with humans gating the final promotion. CI/CD ships the pipeline code; continuous training runs the pipeline to produce new model versions. It is what closes the lifecycle loop in a mature system.

Q4. What is training-serving skew and what are its common causes?

It is a mismatch between the features a model was trained on and the features computed at serving time; the model then receives inputs from a different distribution than it learned and its output is meaningless regardless of model quality. Listing 1 measures three causes on the same model: recomputing normalization statistics per serving batch instead of persisting the training transform (0.828 to 0.694), a unit/parsing bug on one feature (to 0.602), and a stale feature served from a stopped cache (to 0.679). All are plumbing failures, not modelling errors.

Q5. How does a feature store prevent training-serving skew?

Its defining property is a single feature-computation path shared by training and serving: a feature is defined once and both the offline training job and the online serving path read the same definition, so the trained-on value and the served-on value are identical by construction. It provides an offline store (historical values for training sets, with point-in-time-correct joins) and an online store (a low-latency key-value store serving the current feature vector in single-digit milliseconds), plus feature versioning, reuse across teams, and freshness monitoring.

Q6. What is a point-in-time-correct join and why does it matter?

When building a training row for a label event, a point-in-time join includes only feature values that existed *before* that event. Without it you leak the future: joining in a feature computed from data that arrived after the outcome inflates offline metrics and collapses in production, because at serving time that future value does not yet exist. The offline store of a feature store enforces this, which is a large part of why it exists.

Q7. Why is pickling a model risky, and what is the robust alternative?

Pickle stores a reference to the class, not the math, so loading requires the exact class definition and compatible library versions; under version drift it fails to load (Listing 2: `AttributeError`) or, worse, silently mis-predicts. It is also a remote-code-execution vector, since unpickling runs arbitrary code. The robust alternative is to serialize the computation graph (operators plus weights) in a framework-independent format like ONNX, replayed by a minimal runtime that depends on none of the training code — Listing 2 reproduces predictions to a maximum difference of 0.0.

Q8. When are framework-native formats fine, and what is safetensors?

Framework-native formats — PyTorch `state_dict` via `torch.save`, TensorFlow `SavedModel` — are fine within a matched environment and are what most training checkpoints use; the risk appears when the loading environment drifts from the saving one. `safetensors` is a format designed to serialize weights *without* pickle's arbitrary-code-execution risk, so it is the safe choice for sharing or loading untrusted weight files.

Q9. Compare FastAPI, TorchServe, and Triton for serving.

FastAPI (or Flask) wrapping a model is the simplest real option — full control, good for low-to-moderate traffic, and what most first deployments are. TorchServe and TensorFlow Serving are purpose-built model servers that add versioning/hot-swap, multi-model hosting, metrics, and dynamic batching out of the box. Triton is the heavyweight: any framework (ONNX, TensorRT, PyTorch, TF), dynamic batching, concurrent model execution on one GPU, and ensemble pipelines. The complexity earns its place mainly through dynamic batching and GPU concurrency.

Q10. What is dynamic batching and what does it trade off?

The server waits a short window to accumulate multiple requests, then runs them in one forward pass, amortizing the large fixed per-call GPU overhead over many items. Listing 3: one-at-a-time serving caps at 241 req/s and diverges under an 800 req/s load (58 s latency), while batching up to 8 with a 5 ms wait reaches 1,319 req/s at 8 ms latency, and a 20 ms/32 window reaches 2,914 req/s at 19 ms. The tradeoff is explicit — a larger batch and longer wait buy throughput at the cost of latency.

Q11. Why containerize an ML model, and what is ML-specific about it?

A model depends on an exact stack — CUDA, cuDNN, framework build, NumPy, dozens of transitive libraries — and "works on my machine" is the default failure when any drifts. A Docker image freezes the whole userspace into an immutable artifact so the tested environment is the served environment. ML-specific practices: pin a working CUDA/framework base image, use multi-stage builds to keep gigabyte-scale images small (pull latency matters at autoscale), and usually mount or pull model weights at startup rather than baking them in, so one image serves many model versions.

Q12. Why keep model weights out of the container image?

Baking weights into the image ties every model update to an image rebuild and bloats the image, which slows the pull that happens on every autoscale event and node placement. Mounting or pulling weights at startup from object storage or the model registry lets one immutable image serve many model versions, makes rollback a config change rather than a rebuild, and keeps the image small.

Q13. What role does Kubernetes play in ML serving?

It is the orchestration layer above containers: it schedules replica containers across nodes, restarts failed ones, manages rollouts, and drives horizontal autoscaling on load. You do not administer it to discuss ML deployment, but it is where "run N replicas, keep them healthy, scale them on demand" actually happens, and GPU scheduling and custom-metric autoscaling live there.

Q14. What does MLflow provide for model management?

Tracking logs every run's parameters, metrics, code version, and output artifact so experiments are comparable and reproducible. The Model Registry is the system of record for production-bound models: a versioned, named entry that moves through stages (Staging, Production, Archived) with an audit trail, which makes rollback a first-class operation — production is a pointer to a registry version. It also holds lineage: which data and run produced an artifact.

Q15. What is DVC and what problem does it solve?

Git versions code well and large binary data/models badly. DVC (Data Version Control) versions large datasets and model files by storing a small hash-pointer file in Git while the bytes live in object storage, so checking out an old commit restores the exact data and model that commit was built with. It also expresses pipelines as dependency graphs, re-running only stages whose inputs changed. Together: Git for code, DVC for data, a registry for models.

Q16. How does rollback work in a well-run deployment?

Production points at a registry model version, so rollback is repointing it at the previous known-good version — a config change, not a redeploy scramble. Because the image usually does not contain the weights, and because code/data/model are all versioned, the previous state is fully reproducible. This is why the registry's stage transitions and the separation of weights from image matter operationally.

Q17. Distinguish data drift from concept drift.

Data drift (covariate shift) is a change in the input distribution $P(x)$; concept drift is a change in the relationship $P(y|x)$, where the same inputs now map to different outcomes. The operational difference is detectability: data drift is catchable without labels (compare input distributions), while concept drift requires an outcome signal. Listing 5 shows an input-drift detector reading essentially zero (PSI 0.007 to 0.004) while the error rate jumps 0.216 to 0.655 — concept drift is invisible to input monitors.

Q18. How do you detect data drift, and what are the PSI thresholds?

Compare the current feature distribution to a training reference with the Population Stability Index (a symmetrized divergence over fixed training-quantile bins; rule of thumb below 0.1 stable, 0.1 to 0.25 moderate, above 0.25 significant) and the two-sample Kolmogorov-Smirnov statistic for continuous features, plus null-rate and summary-statistic monitors. Listing 4 drifts a feature and PSI climbs 0.002 to 2.06, KS 0.018 to 0.56, crossing 0.25 while accuracy has barely moved — a label-free leading indicator.

Q19. How do you detect concept drift given that labels are often delayed?

Concept drift needs an outcome-derived signal: monitor the error rate once labels arrive with a sequential detector — DDM/EDDM on the error rate, ADWIN's adaptive window, or CUSUM (Listing 5 alarms 32 requests after the true change, where a per-window control chart would false-alarm). The binding constraint is label latency: if labels take weeks (chargebacks, churn) detection is weeks-delayed, so teams also watch label-free proxies (the drift of the output-score distribution and of the inputs) and pay for human-labelled sample streams.

Q20. Does data drift always mean the model has degraded?

No, and saying so is a strong signal. If $P(y|x)$ is unchanged and the model generalizes, the input distribution can move without hurting accuracy — a drift alarm is a prompt to investigate, not proof of degradation. The danger case is drift into a region the model learned poorly, where accuracy does fall (Listing 4's setup). This is exactly why you monitor outcomes and slices too, not just input distributions.

Q21. What should a production monitoring setup actually watch?

Layered signals: input data drift (PSI/KS, null rates, schema violations, feature freshness) as a label-free early warning; prediction-distribution drift (has the mean score or class rate moved?); accuracy and concept-drift detectors once labels arrive; slice metrics on the segments that matter, so an aggregate hiding a subgroup collapse is caught; and operational metrics (latency p50/p99, throughput, error rate, saturation). The mature setup ties an alarm to an action — investigate, then retrain and stage a rollout.

Q22. What is a shadow deployment, and what can it and can it not catch?

The new model receives a copy of every live request and its predictions are logged and compared to production, but never served — so its defining property is zero user harm (Listing 6). It validates the serving path end to end (loads, feature resolution, latency), checks prediction sanity versus the incumbent, and once labels arrive measures the accuracy regression. What it cannot measure is the behavioural response — because users never act on its predictions, it tells you nothing about clicks, conversions, or revenue.

Q23. What does a canary deployment buy, and what is the honest limit?

Serving the new model to 1 to 5% of real traffic behind monitoring with auto-rollback bounds the blast radius — Listing 6 caps exposure at 4.8% versus 100% for a blind cutover. The honest limit, which interviewers probe: the *count* of bad predictions before rollback is similar to a blind cutover (20 vs 18), because detection is set by the statistical test, not the traffic split. What the canary caps is how much of the userbase is exposed *at once* — the real protection against failures a metric cannot pre-see, like a crash loop or latency blowup.

Q24. How do shadow, canary, and A/B testing compose?

They form a ladder answering different questions. Shadow proves the plumbing works at zero user risk but cannot measure behaviour. Canary bounds operational risk while gathering live user feedback on a small slice. A/B testing is the only rung that proves *value* — randomized control/treatment with a significance test on business metrics. In practice: shadow, then canary, then A/B, then a progressive rollout (5 to 25 to 50 to 100%) with monitoring throughout and a registry rollback pointer ready.

Q25. Compare batch, real-time, and streaming inference.

Batch precomputes predictions on a schedule from a snapshot and serves them by lookup — cheapest and simplest, correct when within-day freshness is not needed, but features can be up to a day stale (Listing 7: accuracy 0.822). Real-time computes on demand with the freshest features and request-time context (recovers 1.000) at ~20x per-request cost and a tens-of-ms latency path. Streaming updates features continuously via an event pipeline (Kafka/Flink) for seconds-to-minutes freshness, recovering near-full accuracy at a fraction of real-time cost. Required freshness selects the mode.

Q26. How does feature staleness affect accuracy, and what is the hybrid pattern?

Accuracy falls monotonically with staleness when the label depends on current state — Listing 7 traces 1.000 at 0 h stale to 0.921 at 2 h, 0.844 at 12 h, 0.822 at 24 h. This curve, not fashion, sets the mode: batch where a day-old feature is fine, streaming where minutes matter, real-time where the request carries the signal. The common hybrid precomputes expensive features in batch or streaming and computes only cheap request-time features online, combining them at serving.

Q27. Why can't you run a model server near 100% utilization?

Queueing theory: latency scales like $1/(1-\rho)$, so it detonates as utilization ρ approaches 1 rather than degrading gracefully. Listing 8's M/M/c server reads p99 of 95 ms at 50% utilization, 164 ms at 85%, 275 ms at 92%, and 622 ms at 98%. The headroom that looks like waste is what bounds tail latency; the target utilization is a direct function of your tail-latency SLO and your service-time variance.

Q28. How should an autoscaler be configured for model serving?

Scale on a signal that tracks the SLO — request queue depth, latency, or GPU utilization via a custom metric — not CPU utilization, which is nearly useless for GPU- or IO-bound serving. Listing 8's autoscaler holds ~50% utilization through a 30 to 210 req/s diurnal swing, meeting a 150 ms p99 SLO for 42% fewer replica-hours than a peak-provisioned fleet. Caveats: cold replicas take minutes (image pull, model load, GPU warm-up) so keep headroom and use scheduled/predictive scaling for spikes; set a maximum to bound cost and a minimum to hold a warm floor.